Robot Car – CV

Updated Nov 12, 2026 (app 4.2) · also as a PDF



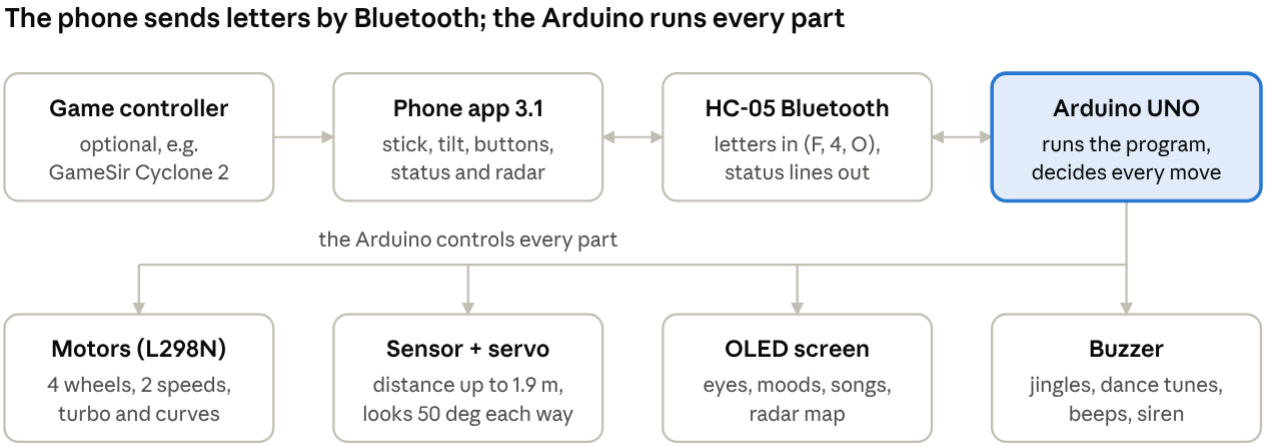
Profile

A 4-wheel-drive Arduino robot car that drives from a phone, a computer or a game controller with smooth RC-car steering, avoids obstacles, maps the room with radar, turns its sensor head on command, plays six games and dances to 39 songs with real choreography. Built by Muhammad Ayaan Malik, from a starter tutorial (version 0) to version 9 plus play modes, radar, smooth steering, its own Android app with a live 3D car, and a web remote (November 2026).

Name	Robot Car (program: Robot_Car_Main, v9 + play modes + radar + steering + head)
Brain	Arduino UNO (ATmega328P, 32 KB program memory, 2 KB RAM) on a Sensor Shield V5
Drive	4 DC motors through an L298N driver; each side at its own speed, 31 steps forward and 31 backward
Senses	HC-SR04 ultrasonic distance sensor on an SG90 servo (looks 50° left and right, by itself or where you point it)
Shows	128 x 64 SSD1306 OLED screen: animated eyes, mode, distance, song, radar map
Sounds	Passive buzzer: jingles, dance tunes, parking beeps, horn, siren
Talks	HC-05 Bluetooth at 9600 baud, to its own Android app (4.2) and a web remote for computers
Power	3 x 18650 cells (motors + Arduino), 4 x AA (servo)
Skills	Smooth steering, head control, 6 self-driving modes, 6 play modes, radar, 39 dances

Program size	31,136 bytes (96.5% of memory); song versions up to 99.2%
Tested	Car simulator: 114 of 114 checks on the newest main program; all 39 dances 847 of 847 one step earlier. App: 256 checks pass
Code	Public on GitHub: mayaanmalik2486/robot-car

How it works



Every button becomes one letter sent over Bluetooth; the steering wheel, lever and sticks become number codes that set the speed of each side of the car, and the head wheel becomes codes that point the servo. The Arduino turns them into motor moves, sounds, faces and sensor sweeps, and sends back what it is doing, its song, what it understands and radar readings.

Hardware and wiring

Every part plugs into the Sensor Shield V5, where each pin has a row of three: signal (S), +5 V (V) and ground (G).

Part	Job	Pins
Arduino UNO + Sensor Shield V5	Runs the program, connects everything	–
L298N motor driver	Drives the wheels; its 5 V output powers the Arduino	ENA D5 (left speed), ENB D6 (right speed), IN1 D2, IN2 D4 (left direction), IN3 D7, IN4 D10 (right direction)
4 DC motors	Two on each side, wired side by side	OUT1 + OUT2 left, OUT3 + OUT4 right
HC-05 Bluetooth	Letters in from the phone, status out	TXD → D3 (Arduino listens), RXD → A0 (Arduino talks)
HC-SR04 ultrasonic sensor	Measures distance up to about 1.9 m	TRIG D11, ECHO D12

Part	Job	Pins
SG90 servo	Turns the sensor: 40° = right, 90° = ahead, 140° = left	D9
SSD1306 OLED, 128 x 64	Eyes, text, radar picture	SDA A4, SCL A5 (I2C at 400 kHz, address 0x3C or 0x3D)
Passive buzzer board	All sounds, made by a tone generator on Timer2 (about 19.8 kHz)	A1
IR receiver (add-on only)	TV-style remote	D8

Free pins: A2 and A3 (A3's noise seeds the random numbers for chaos mode). The USB pins D0 and D1 stay free for uploading.

Power

Two separate battery packs keep the servo's current spikes away from the Arduino.

- **Main pack: 3 x 18650 cells** → on/off switch → L298N motor input. The L298N's 5 V regulator (jumper ON) feeds the Arduino's 5 V pin; the grounds are joined.
- **Servo pack: 4 x AA** (4 x 1.2 V NiMH works even better: 4.8 V is the SG90's normal voltage) → straight to the servo's red wire. Its minus joins the shield's ground at A2.
- **Battery check:** the Arduino measures its own supply against its built-in 1.1 V reference every 2 s. It warns when the supply drops 300 mV below its starting value or below 4,300 mV for 10 s. Because the regulator holds 5 V until the pack is nearly flat, the warning comes late.

Driving skills

The car drives while a control is held and stops the moment it is let go. There are two ways to drive it.

Smooth steering (app 4.0 and newer, the web remote). The app sets the speed of the left wheels and the right wheels separately, from -31 (full backward) to 31 (full forward).

You do	The car does
Throttle only	Both sides the same speed: straight on. The further you push, the faster (motor power 95 to 255)
Throttle + a little steering	The inner side slows: a wide bend
Full throttle + full steering	The inner side runs backward at 30%: the tightest bend
Steering only	The sides run opposite ways at 85%: spin on the spot
Reverse + steering	Backs up while turning
Let go	Both sides 0, then stop

The car still refuses to drive forward into a wall, and it stops by itself if no wheel speed arrives for 0.6 s (the app repeats them every 0.2 s), so walking out of range stops it.

Letters (older apps, any Bluetooth RC app). Unchanged, so old apps still work:

Move	Letter	How it works
Forward / backward	F / B	Both sides at the current speed
Spin left / right	L / R	Sides turn opposite ways at 85% of the speed, on the spot
Curve left / right	G / H	Forward while turning: the inner wheels run backward at 30% for a sharp bend
Stop	S	Stops everything, including any mode, and centres the sensor head
Fast / slow	W / Q	Full power (255) or 60% (153). Now takes effect at once, also while reversing or spinning
Turbo	T	1.5 s at full power with a whoosh and "TURBO!"; still stops for walls unless football mode is on
Smooth brake	u	Each side lowers its own power by 10 every 40 ms, so a bend stays a bend
Horn	Y	"Beep beeeep" (880 Hz)

Sensor head. Codes 6 to 26 point the servo in 5° steps: 6 = 50° right, 16 = straight ahead, 26 = 50° left. It stays there until the next code, a mode that uses the servo, or stop.

Obstacle stop: driving forward, it measures the distance three times per check (15 ms apart) and keeps the middle value. At 30 cm it stops, giving the wheels a 70 ms burst in reverse so it stops short, and shows surprised eyes for 1 s.

Parking beeps: from 80 cm, 1,650 Hz beeps come faster as it gets closer: every 600 ms far away, every 90 ms when very close.

Self-driving modes

One tap starts them; they keep going until another command arrives. An "S" sent within 1.5 s of the tap (a phone button being let go) is ignored so the mode keeps running.

Mode	Letter	What it does
Auto forward / backward	2 / J	Drives straight by itself and stops at obstacles
Search	I	Stops, backs up 120 ms, looks left, right and ahead with the servo, then turns toward the side with the most room in 60 ms steps (about 20° each) until at least 100 cm is clear, plus one extra step so the whole car clears the edge. Boxed in: turns 90° at a time. If the distance ahead hasn't changed for 1.5 s while driving (stuck on something it can't see), it backs up 300 ms and turns
Chaos	1	Random straight runs (0.4–1.3 s), turns (0.12–0.45 s), a 15% chance of a spin (0.4–0.9 s) and a 15% chance of a reverse wiggle, at 75% speed. Obstacle: backs up 250 ms and turns. Angry eyes

Mode	Letter	What it does
Donut	3	Spins round and round; stopped, it shows dizzy eyes and wobbles its head ±25° for 1.3 s
180° turn	U	Spins exactly 455 ms, tuned to half a turn on this car (a 90° turn is 227 ms)

Play modes

Six games built from the parts already on the car. They always use the distance sensor, even in football mode, and "see" anything closer than 150 cm. Each can be switched off in the code to free memory.

Mode	Letter	How it plays
Follow me	M	Keeps 18–30 cm from your hand or feet at 65% speed: follows past 30 cm, backs off under 18 cm. Nothing within 90 cm: it looks at 5 angles and turns toward the nearest thing
Goalkeeper	K	Scans for the ball, turns to face it, and punches it away (a 220 ms dash forward and back) when it comes within 30 cm
Kick	k	Shoots straight away (starts goalkeeper if needed)
Guard	A	Beeps 5 times (time to walk away), learns the room, then watches. Anything 25 cm closer than the room was = intruder: siren, angry eyes, "INTRUDER", it turns to face them and the phone vibrates. Calms down 5 s after they leave
Music	N	A hand theremin: hand distance 5–60 cm plays notes on a pentatonic scale (closer = higher); farther = silence
Gestures	E	Hand within 35 cm: quick tap (under 450 ms) = spin round, double tap = happy wiggle, held under 12 cm for 700 ms = it backs away, held further for 1.4 s = it comes to you
Pet	P	Wanders at 50% speed, sits and looks around, gets bored after 25 s, falls asleep snoring after 45 s, and purrs with happy eyes when "petted" (a hand within 25 cm)

Radar

The car stands still and sweeps its sensor across 11 directions, 10° apart (50° right to 50° left and back), one step every 160 ms – a full sweep in about 2 s.

- **On the car's screen:** the eyes make way for a radar picture – the car at the bottom, dotted rings at 75 cm and 150 cm, the sweep line, and a 3 x 3 dot for everything within 150 cm. The top row reads "RADAR near 42cm".
- **In the app:** every reading is sent as a line like `#R:f42` (direction a = far right … f = ahead … k = far left, then cm). The app's Radar page draws the same map with rings at 50, 100 and 150 cm; dots fade over 6 s. The 3D car sweeps its head and shows the same dots on the floor round it.
- **Start / stop:** letter O (app Radar page, or START on a controller); any other command stops it.
- **Cost:** about 0.9 KB of program memory; switch HAS_RADAR off to free it.

Dancing

The car dances whole songs in time with the music, each with its own choreography. Every song is a full copy of the program (in the Dances folder) with its own tempo, key, dance steps and routine, so every other skill

still works.

- **Choreography:** signature moves per song and a routine for each part (intro, verse, chorus, drop). The chorus routine returns every chorus, phrases end on a pose, the bar before a big part is a lead-in, the last bar is a finale.
- **Moves:** spins, twists, sways, dashes, looks and freezes, soft (gentler) versions, and "round" spins that go exactly once round and finish facing the front. Every step returns the car to its spot; it never dances forward into anything closer than 30 cm.
- **Music:** the buzzer plays an original tune in the song's key – bass on the beat plus a melody that follows the chords – not the real melody. A few songs have a special moment that flashes text on the screen with a zap sound.
- **Staying in time:** press dance on the first beat; tap again on any beat and it lines itself up without restarting. The screen shows the song name (scrolling if long), the part and the time in the song.
- **Accuracy:** in the simulator, 12,624 of 12,624 moves landed on the beat and 18,288 of 18,288 notes were right.

Song	Artist	Length	Tempo	Key
Aal Izz Well (3 Idiots)	Sonu Nigam, Shaan, Swanand Kirkire	4:36	120	D# major
Aankhen Milaney Waaley	Nazia & Zoheb Hassan	3:34	107.4	F# major
Ambarsariya	Sona Mohapatra	4:08	90	E minor
Baba Bandook	Ali Azmat & Haroon	3:53	88.5	D major
Badtameez Dil	Benny Dayal & Shefali Alvares (Pritam)	4:13	106	D minor
Ben 10 (original theme)	Moxy	1:01	136	D# minor
Ben 10: Alien Force theme	Andy Sturmer	0:31	135	C minor
Ben 10: Ultimate Alien theme	Carter, McCuistion & Ritmanis	0:21	85	C minor
Ben 10: Omniverse theme	Parry Gripp	0:20	98	B minor
Ben 10: Omniverse (kid Ben opening)	Parry Gripp	0:21	98	B minor
Bolve	Madan Maddi & Sukshinder Shinda	5:12	114	E minor
Chammak Challo (Ra.One)	Akon & Hamsika Iyer	3:47	134.3	B minor
Confidence	Kim	3:44	88	C# major
Copacabana	Barry Manilow	4:05	116.6	D# major
Distractions	Haiti Babii	2:32	101	D major
Feet Don't Fail Me Now	Joy Crookes	3:18	113	A# minor
Flashing Lights	Kanye West feat. Dwele	3:57	90.5	F# minor
Get Lucky (Drumless Edition)	Daft Punk	6:09	116	F# minor
God's Plan	Drake	3:19	77	E minor
Greenback Boogie (Suits theme)	Ima Robot	0:31	143	F major
Iraaday	Rovalio & Abdul Hannan	2:13	90	F# major

Song	Artist	Length	Tempo	Key
Kabhi Kabhi Aditi	Rashid Ali (A.R. Rahman)	3:38	95	G minor
Loser	Tame Impala	3:44	83	A# minor
Love Me	JMSN	4:33	81	A# minor
Loverboy	A-Wall	3:45	120	C major
My Destiny (Slowed)	Delinquent feat. KCAT	4:39	106	A minor
My Type	Saint Motel	3:26	118	C minor
National Treasures	Drake	3:20	129.5	G# minor
oh yeah?	Steve Lacy	2:51	73	C# major
Save Me	Remy Zero	4:01	96	E major
Sh-Boom	The Chords	2:26	68.5	F major
the kill 2	Lex Amarni & 2muchmotion	2:01	103	D minor
The Mentalist (main title)	Blake Neely	1:08	105	D minor
The Night Begins to Shine	B.E.R.	3:30	130	E minor
They Don't Care About Us	Michael Jackson	4:44	90	D minor
Touch The Sky	Kanye West feat. Lupe Fiasco	3:57	106.7	B minor
Upside Down	Diana Ross	4:05	108.5	G minor
We Are The People	Empire Of The Sun	4:32	123.4	E minor
worry	LONOWN feat. riserayss	3:01	99	B major

Tempo is in beats per minute. The main program (no song file) dances to its own built-in "Robot Groove" at 120 BPM.

Personality

The screen gives the car a face and the buzzer gives every action a sound.

- **Eyes:** two rounded eyes in the middle six rows of the screen, animated at up to one frame every 60 ms and drawn tile by tile so only changed parts are sent. They blink every 2–4.5 s (150 ms) and look where the sensor looks.
- **Moods:** happy (hello, dance), angry (chaos, guard alarm), fierce (turbo), focused (search), squinting (braking), sleepy (25 s with nothing to do), surprised (emergency stop, 1 s), dizzy with wobbling eyes (after a donut), purring (petted).
- **Screen text:** top row = Bluetooth, speed and warnings (or the song name while dancing); bottom row = what it is doing and the distance in cm. Each row redraws at most 4 times a second.
- **Hello:** at switch-on, and when the phone sends its first letter, it shows "Hello! :-)" / "Phone connected!" for 3 s, plays a fanfare and waves the servo.
- **Sounds:** a jingle for each mode, the dance tunes, parking beeps, horn, turbo whoosh, siren, snoring, a sad low-battery tune. Mute (X) silences everything; press again to unmute.

Safety and battery

- **Obstacle stop at 30 cm** with a reverse jerk, in manual forward, auto forward, curves, turbo and dance.
- **Football mode (V):** the sensor is switched fully off so the car can push a ball or bump into things; search, chaos, radar and the play modes still use it. Press V again to switch it back on.
- **Never left driving:** the app sends stop when you let go, when it goes to the background, or when a controller disconnects; stop is sent twice to be sure. With smooth steering the car also stops by itself after 0.6 s without a wheel speed (phone out of range, app crashed).
- **STOP always works:** the app sends "both sides 0" first, which stops even a mode started less than 1.5 s ago.
- **Loose-wire proof screen:** the I2C bus has a 3 ms timeout, so a loose OLED wire can never freeze the car.
- **Low-battery warning:** "LOW BATT!" on the screen, a sad tune (again every minute) and a phone (and controller) vibration.

Command language

The phone sends single characters over Bluetooth (9600 baud); the car answers with short lines.

Letter	Command	Letter	Command
F / B	Forward / backward	L / R	Spin left / right
G / H	Curve left / right	S	Stop
W / Q	Full / slow speed	T	Turbo
Y	Horn	u	Smooth brake
2 / J	Auto forward / backward	3	Donut
U	180° turn	I	Search
1	Chaos	4	Dance (again on a beat = line up)
M	Follow me	K / k	Goalkeeper / kick
A	Guard	N	Music
E	Gestures	P	Pet
O	Radar	X / x	Mute / unmute
V / v	Football mode on / off	?	"What are you doing?"

Number codes (new):

Code	Meaning
128 to 191	Left wheels: speed = code - 160 (so 129 = full backward, 160 = stop, 191 = full forward)
192 to 255	Right wheels: speed = code - 224
6 to 26	Sensor head: angle = (code - 16) x 5°, plus = left

These can never be mixed up with a letter. The app sends a wheel code only when the speed changes (at most every 60 ms), repeats both every 0.2 s while driving, and sends a head code only when it changes (at

most every 70 ms).

The car's answers:

Line	Meaning	When
`~DRIVE|0010`	What it is doing, then 1/0 for muted, football, slow, low battery	When something changes (at most 4 times a second, never mid-dance or mid-turn) and after "?"
`#S:Badtameez Dil`	The song this program dances to	After "?"
`#V:3`	It knows the steering codes (2) and the head codes (3)	After "?"
`#R:f42`	A radar reading: direction a–k, distance in cm (999 = nothing)	Every 160 ms in radar mode

Repeated drive letters are ignored (some apps repeat while a button is held), and the same mode letter again does not restart a running mode. The car now reads every waiting character on each pass instead of one.

The phone app

Robot Car 4.2 is the car's own Android app: about 105 KB, Android 5 and newer, free to download from one link and shareable from inside the app. It is held sideways. Every earlier version (1.0 to 4.1) is kept in `App/Old_APKs`.



Area	What is there
Top bar	Connect, the song the car dances to, the connected controller, top speed (fast / slow), driving style (wheel / 1 stick / tilt), settings, help
Left	The throttle lever (or the thumbstick)

Area	What is there
Middle	A live 3D car with the status over it; STOP and four action bubbles (dance, kick, horn, turbo); four pages: Moves, Play, Radar, Head
Right	The steering wheel (with 1 stick or tilt: STOP and the action bubbles)

- **Wheel driving (default):** a throttle lever (up = forward, down = backward, as fast as you push) and a steering wheel you grab anywhere and turn, 126° each way for full steering. Both spring back when let go.
- **1 stick and tilt:** one thumbstick, or hold the pad and tilt the phone like a steering wheel (25° = full).
- **Live 3D car:** a small 3D engine written for this app. The wheels spin, the floor moves and turns, the car leans in bends, bounces when it dances, sweeps its head in radar mode with the dots on the floor, and meets a red wall when blocked. Without a connection it follows the controls, so everything can be tried first.
- **Head page:** a blue wheel turns the sensor head up to 50° each way; it stays where it is left. LOOK STRAIGHT or STOP centres it.
- **Bubble buttons:** glossy 3D keys with a shine, a glow of their own colour, a ripple and a springy press; the button of the running mode glows.
- **Any screen:** the layout re-arranges itself for small phones, tablets, upright and resized windows (checked at 31 sizes).
- **Settings:** driving style, top speed, steering feel, dead zones, haptics, controller badges, reset.
- **Live status and alerts:** guard intruders and low battery pop up a message and vibrate the phone and controller; a blocked car gives a short tick.
- **Smart connecting:** pick the HC-05 once; the app reconnects by itself, offers Reconnect when the link is lost, and keeps the screen on.
- **Old-program check:** if the car doesn't answer within 3 s the app warns "OLD CAR PROGRAM"; if it answers without `#V` the app drives with the old letters instead.
- **Link log:** hold the 3D picture to see everything sent and every line the car answered.
- **Built from scratch:** plain Java with no extra libraries, about 20 small classes, signed with its own key so each version installs over the last.

The web remote

The same remote as a web page for Chrome or Edge on a computer: the same layout, 3D car, wheel, lever and controller map. It talks to the HC-05 through Web Serial and drives with the mouse, the keyboard (W A S D, Space = stop) or a controller. Its driving logic is a JavaScript copy of the app's and is checked to give exactly the same output. Open `Web/index.html`.

Game controller

A controller such as the GameSir Cyclone 2 drives the car through the phone (the HC-05 can't pair with a controller directly). Connect it by Bluetooth (Android mode, yellow light) or its 2.4 GHz dongle with a USB-C adapter (green light). It drives like an RC transmitter: throttle on one thumb, steering on the other. The pressed button lights up on screen, and every button can be changed.

Controller	Does
Left stick up / down	Throttle: the further you push, the faster
Right stick left / right	Steering (no throttle = spin on the spot)
RT / LT	Turbo / smooth brake
A / B / X / Y	Dance / STOP / kick / horn
RB	Football mode
LB	SHIFT (hold for the second jobs)
D-pad up / right / down / left	Search / chaos / donut / 180° turn
START / SELECT	Radar / mute
L3 / R3 (click the sticks)	Auto forward / auto backward
Hold LB + A / B / X / Y	Follow me / guard / goalkeeper / pet
Hold LB + D-pad up / down	Music / gestures
Hold LB + D-pad left / right	Top speed slow / fast
Hold LB + right stick	Turn the sensor head

The app understands all four Cyclone 2 modes, shows a live drawing of the controller with stick and trigger values, remembers a separate button map for each controller, and stops the car if the controller disconnects. It rumbles for an intruder (long), low battery (medium), a blocked car (tick) and turbo (pulse). The back buttons M1 / M2 can copy any button through the GameSir app.

Software

The car runs one Arduino program of about 4,500 lines, written in plain language with a comment on almost every line.

Program layout: 1 libraries · 2 pins · 3 settings · 3b dance moves · 3c the song (tempo, steps, choreography) · 5 memory · 6 motors · 7 buzzer · 8 screen · 9 distance sensor · 10 servo · 11 greeting · 12 timed moves · 13 stop/start helpers · 15 Bluetooth · 16 self-driving modes · 16b play modes, radar, turbo, battery · 17 setup and loop.

How it runs: almost no waiting (only 15 ms between distance pings); every mode is a step machine checked on each pass of the loop (about every 50 ms, set by the distance measurement). Timed moves (180° turn, play-mode moves, dance spins) stop on the exact millisecond, and the screen pauses drawing just before a dance step so moves stay on the beat. Bluetooth is read twice per loop, and every waiting character is handled each time.

Memory	Used	Of
Main program	31,136 bytes (96.5%)	32,256
Fullest song file (Get Lucky)	31,988 bytes (99.2%), 268 bytes left	32,256
RAM (variables)	992 bytes (48%)	2,048

Smooth steering, the four fixes and the head codes cost 480 bytes together.

Switches (section 3): HAS_FOLLOW, HAS_KEEPER, HAS_GUARD, HAS_MUSIC, HAS_GESTURE, HAS_PET, HAS_RADAR turn each mode on or off; debug messages are off (saves 2.9 KB).

Libraries: U8g2 (screen), Servo, SoftwareSerial (Bluetooth), Wire (I2C).

Add-ons and tools:

- **IR remote add-on:** one file + 3 lines puts a TV-style remote back (radar off, and pet off in the bigger song files, to make room).
- **UPDATE_ALL_SONGS.bat:** copies a change in the main program into all 39 song files (each keeps its song and choreography) and checks all 40 compile.
- **Every sketch opens ready for Arduino Uno on COM4.**

Test record

Every change is checked on the computer. The car program runs in a simulator: the real compiled program on a simulated ATmega328P, with a simulated distance sensor, servo, motors, buzzer, screen and Bluetooth. The app's logic and drawing code run as plain Java.

What is checked	Result (Nov 12, 2026)
Car simulator, newest main program (steering, head codes, safety stop, fixes, every mode, the full demo dance)	114 of 114 passed, 0 failed
Car simulator, main program + all 39 song dances (run on Nov 10 with steering, before the 44-byte head code was added)	847 of 847 passed, 0 failed; the re-run with the head code was still going when this was written
Programs compiled and fitting in memory	40 of 40 (main + 39 songs)
App driving logic, controller map, button presses, car answers	131 of 131
App layout: nothing overlapping or cut off at 31 screen sizes x 3 driving styles	93 of 93
App drawing code run and drawn to pictures; lever and wheel driven with pretend fingers	21 of 21
Web remote gives the same layout, wheel speeds and codes as the app	11 of 11
Every Android call checked against Android 5	All newer calls are behind a version check

The simulator tests cover smooth steering, the 0.6 s safety stop, the head codes, the speed and brake fixes, a drive replayed from the app's real logic, curves, turbo, horn and status, every play mode, radar, football mode, mute, stopping each mode, the 180° turn, faces and dizziness, being stuck with no echo, dancing next to a wall, tapping dance back in time, and special song moments.

Not yet tested: the app on a real phone, the new car program on the real car, and the GameSir Cyclone 2 in hand.

History

The car grew from a tutorial obstacle avoider into a phone-controlled entertainer. Every version is kept in the Old_Versions folder.

When	Version	What changed
Nov 11, 2026	App 4.2 + head	A wheel that turns the sensor head (new car codes 6–26), on the controller with LB + right stick
Nov 11, 2026	App 4.1	The new look: live 3D car, steering wheel and throttle lever, glossy bubble buttons, animations
Nov 10, 2026	App 4.0 + steering + web	Smooth steering (the app sets each side's speed), 0.6 s safety stop, four car fixes, GameSir Cyclone 2 map with remapping, settings, controller screen, web remote for computers
Oct 20, 2026	App 3.1	Spots an old car program, speed button without status, link log, safer screen tap
Oct 20, 2026	Radar + app 3.0	Radar mode, song name to the app, tilt driving, Moves / Play / Radar pages, pressable buttons, racing-style controller map
Oct 20, 2026	Guide	Full guide to everything the car and app can do (page + PDF)
Oct 19, 2026	App 2.0	Game-pad layout in landscape, emoji buttons, game controller support
Oct 19, 2026	Play modes + app 1.0	Follow me, goalkeeper, guard, music, gestures, pet, turbo, horn, low-battery check, status line, own Android app, IR remote add-on, update-all-songs tool
Oct 19, 2026	Football mode	V switches the distance sensor fully off
Oct 19, 2026	Curve	"Drift" renamed curve, sharper (inner wheels backward at 30%)
Oct 19, 2026	Choreography	Real choreography for every song
Oct 18, 2026	Dances	38 song dances, then Badtameez Dil (39)
Oct 16, 2026	v8	Eyes, search, chaos, dance; first upload to GitHub
Earlier	v7.1 "Robot Original"	The version the big upgrade started from
Earlier	v7 "Asura"	v7 final
Earlier	v5.5 – v6	Better IR remote, first Bluetooth, part Bluetooth
Earlier	v5	Whole IR remote; a cliff-sensor prototype that did not work
Earlier	v1 – v4	Turn and move ahead, turn on its axis, obstacle detection, better obstacle detection

When	Version	What changed
Start	v0	Starter tutorial code (Science N Music Studio), servo test, first obstacle avoiders

Limits and future plans

Limits today:

- **Memory is nearly full** (96.5%, up to 99.2% in song files; the fullest has 268 bytes left): new features need something switched off first.
- **Android and computers only:** iPhones can't talk to an HC-05 module, in an app or a web page.
- **Battery:** it can only warn when nearly empty; a real percentage needs two resistors (a voltage divider into A2).
- **One sensor, straight ahead:** thin, soft or angled things (chair legs, cushions) can be missed.
- **Half-duplex Bluetooth:** while the car sends a line, a letter arriving at that moment can be missed.
- **Old programs:** the songs in the old Robot_Dances folder and the v7.1 sketch don't know the play modes, radar or status line; any program from before Nov 10 doesn't know smooth steering, and before Nov 11 the head codes. The app falls back to letters for them.
- **Not tried on real hardware yet:** app 4.x, the steering and the head codes are checked on the computer only.

Ideas saved for later: record and replay a route (kept after switch-off), more emoji faces and sound effects (police siren, reversing beep, laser), screen games (reaction game, Simon says), a wall follower for mazes, a line follower or table-edge stop (with the old sensors on free pins A2/A3), and a real battery meter.

Where to find everything

All the code, songs, app and guides are public in one GitHub repository.

What	Where
Everything	github.com/mayaanmalik2486/robot-car
The app (download)	RobotCar.apk
App guide	App/README.md
Web remote	Web
Full guide with this week's changes	CAR_AND_APP_GUIDE.md
Main program	Robot_Car_Main
Wiring and power guide	POWER_AND_WIRING.md
The 39 song programs	Dances
IR remote add-on	IR_Remote_Addon
Every old version	Old_Versions
Ideas and memory budget	IDEAS.md

Scan with a phone camera

Download the app	GitHub repository	App guide
		

This CV	Full guide	The 39 songs
		